1. Problem Title & Link

Title: **LeetCode 1011: Capacity To Ship Packages Within D Days**

Link: https://leetcode.com/problems/capacity-to-ship-packages-within-d-days/

2. Problem Statement (Short Summary)

You are given:

- An array weights[] where weights[i] represents the weight of a package.
- An integer days representing the number of days to ship all packages.

Rules:

- Packages must be shipped **in the same order** as given.
- Each day, the ship can carry packages **up to a maximum weight capacity**.
- You cannot split a package.

Goal:

Find the **minimum ship capacity** required to ship all packages within days.

3. Examples (Input → Output)

Example 1

Input

weights = [1,2,3,4,5,6,7,8,9,10]

days = 5

Output

15

Explanation

One valid shipment schedule:

Day 1 → 1+2+3+4+5 = 15

Day 2 → 6+7 = 13

Day 3 → 8

Day 4 → 9

Day 5 → 10

Minimum capacity required = **15**

Example 2

Input

weights = [3,2,2,4,1,4]

days = 3

Output

6

Explanation

Day 1 → 3+2
Day 2 → 2+4
Day 3 → 1+4

Example 3

Input

weights = [1,2,3,1,1]

days = 4

Output

3

4. Constraints

1 ≤ weights.length ≤ 50,000

1 ≤ weights[i] ≤ 500

1 ≤ days ≤ weights.length

Observations:

- Large input size
- Need an efficient approach
- Capacity must be **at least the maximum weight**

5. Core Concept (Pattern / Topic)

Binary Search on Answer

Why?

We are searching for the **minimum capacity** that satisfies the condition.

Binary search works when:

- The answer space is **monotonic**
- If capacity works → larger capacity also works.

6. Thought Process (Step-by-Step Explanation)

Step 1 — Brute Force Idea

Try every possible capacity.

Range:

min = max(weights)

max = sum(weights)

For each capacity:

Simulate shipping process.

Time complexity becomes too slow.

Step 2 — Key Observation

If a capacity C can ship packages in ≤ days, then:

any capacity > C will also work

This creates a **monotonic property**.

Example

Capacity → Days Needed

10 → 7 days ❌

12 → 6 days ❌

15 → 5 days ✅

20 → 4 days ✅

We need the **first capacity that works**.

This is a perfect **binary search scenario**.

7. Visual / Intuition Diagram (ASCII)

Search space

```
max_weight              total_weight
   |--------------------------|
   10    12    15    20
   ❌    ❌    ✅    ✅
```

Goal

Find first valid capacity

Binary search helps locate the boundary.

8. Pseudocode (Language Independent)

```
low = max(weights)
high = sum(weights)

while low <= high:

   mid = (low + high) // 2

   if canShip(mid):
      high = mid - 1
   else:
```

```
        low = mid + 1

return low
Helper function:
canShip(capacity):

    days_needed = 1
    current_load = 0

    for weight in weights:

        if current_load + weight > capacity:
            days_needed++
            current_load = 0

        current_load += weight

return days_needed <= days
```

9. Code Implementation

Python

```
class Solution:
    def shipWithinDays(self, weights, days):

        def canShip(capacity):
            days_needed = 1
            current = 0

            for w in weights:
                if current + w > capacity:
                    days_needed += 1
                    current = 0
                current += w

            return days_needed <= days

        left = max(weights)
```

```
    right = sum(weights)

    while left <= right:

        mid = (left + right) // 2

        if canShip(mid):
            right = mid - 1
        else:
            left = mid + 1

    return left
```

Java

```
class Solution {

    public int shipWithinDays(int[] weights, int days) {

        int left = 0, right = 0;

        for(int w : weights){
            left = Math.max(left, w);
            right += w;
        }

        while(left <= right){

            int mid = left + (right-left)/2;

            if(canShip(weights, mid, days)){
                right = mid - 1;
            }
            else{
                left = mid + 1;
            }
        }
```

```
        return left;
    }

    private boolean canShip(int[] weights, int capacity, int days){

        int daysNeeded = 1;
        int current = 0;

        for(int w : weights){

            if(current + w > capacity){
                daysNeeded++;
                current = 0;
            }

            current += w;
        }

        return daysNeeded <= days;
    }
}
```

10. Time & Space Complexity

Time Complexity

Binary search iterations:

log(sum(weights))

Each iteration scans array:

O(n)

Total:

O(n log(sum(weights)))

Space Complexity

O(1)

Only variables used.

11. Common Mistakes / Edge Cases

1. Starting binary search from 0 instead of max(weights)

2. Forgetting package order constraint
3. Incorrect mid calculation
4. Not resetting current load properly
5. Off-by-one errors in binary search

12. Detailed Dry Run

Example

weights = [3,2,2,4,1,4]

days = 3

Search range

low = 4

high = 16

Iteration 1

mid = 10

Shipping

Day1 → 3+2+2 = 7

Day2 → 4+1 = 5

Day3 → 4

Works.

Search smaller capacity.

Iteration 2

mid = 6

Shipping

Day1 → 3+2

Day2 → 2+4

Day3 → 1+4

Works.

Iteration 3

mid = 5

Fails.

Final answer

6

13. Common Use Cases (Real-Life / Interview)

Binary Search on Answer is used in:

- Load balancing systems
- Server capacity allocation
- Resource scheduling
- Manufacturing throughput planning

14. Common Traps (Important!)

1. Confusing **binary search on index vs answer**
2. Using incorrect lower bound
3. Not recognizing monotonic condition
4. Wrong simulation logic

15. Builds To (Related LeetCode Problems)

Binary Search on Answer progression

LC 875 Koko Eating Bananas

LC 1011 Capacity To Ship Packages

LC 410 Split Array Largest Sum

LC 1482 Minimum Days to Make Bouquets

LC 1283 Smallest Divisor Given Threshold

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Brute Force	O(n * sum(weights))	Too slow
Binary Search	O(n log(sum))	Optimal

17. Why This Solution Works

The number of days required is **monotonically decreasing as capacity increases**.

This monotonic behavior allows binary search to efficiently find the **minimum valid capacity**.

18. Variations / Follow-Up Questions

Possible interview follow-ups:

1. Minimize **maximum load per day**
2. Split array into **k subarrays**
3. Optimize shipping with **variable truck sizes**
4. Allow **package reordering**